

Exp No: 13 EMBEDDED C PROGRAM TO ADD AN ARRAY OF 16-BIT NUMBERS AND STORE THE 32-BIT RESULT IN INTERNAL RAM

AIM:

To write an Embedded C program to add an array of 16-bit numbers and store the results in internal RAM using Keil software.

SOFTWARE REQUIRED:

- Keil

PROGRAM:

```
#include <reg51.h>

void main(void)
{
    unsigned int i;
    unsigned int array[5] = {0x1111, 0x2222, 0x8888, 0x4444, 0xABCD};
    unsigned long int sum = 0;

    for(i = 0; i < 5; i++)
    {
        sum = sum + array[i];
    }

    P0 = (unsigned char)(sum & 0xFF);      // Lower 8 bits
    P1 = (unsigned char)((sum >> 8) & 0xFF); // Next 8 bits
    P2 = (unsigned char)((sum >> 16) & 0xFF); // Upper 8 bits
    P3 = 0x00;                          // Highest byte (not needed here)

    while(1);
}
```

OUTPUT

The screenshot displays the Keil uVision IDE interface. The main window shows the disassembly of the program, with the following assembly code:

```
10: {
11:     sum = sum + array[i];
12:     MOV     A, R3
13:     ADD     A, ACC(0xE0)
14:     ADD     A, #0x08
15: }
```

The C code is also visible, showing the main function and the calculation of the sum:

```
1 #include <reg51.h>
2
3 void main(void)
4 {
5     unsigned int i;
6     unsigned int array[5] = {0x1111, 0x2222, 0x8888, 0x4444, 0xABCD};
7     unsigned long int sum = 0;
8
9     for(i = 0; i < 5; i++)
10    {
11        sum = sum + array[i];
12    }
13
14    P0 = (unsigned char)(sum & 0xFF); // Lower 8 bits
15    P1 = (unsigned char)((sum >> 8) & 0xFF); // Next 8 bits
16    P2 = (unsigned char)((sum >> 16) & 0xFF); // Upper 8 bits
17    P3 = 0x00; // Highest byte (not needed here)
18
19    while(1);
20 }
```

The registers window shows the state of the processor registers, including R0, R1, R2, R3, R4, R5, R6, R7, and the PC (Program Counter).

The command window shows the simulation status, including the command "Running with Code Size Limit: 2K" and the load path "Load 'C:\\Keil_v5\\CS1\\Examples\\HELLO\\Objects\\EXP_13'".

The call stack window shows the current function call stack, including the main function and the array variable.

The simulation status bar at the bottom shows the simulation time (t1: 188.11862300 sec), the current location (L20 C:2), and the current state (CAP NUM SCRL OVR: R/W).

RESULT:

Thus the program has been successfully verified and executed.